Auditoría del módulo de sanidad y veterinaria

Fecha: **19 de octubre de 2026**. Base revisada: árbol de trabajo existente, incluidos cambios locales; HEAD `e7bffc6`. Instrucciones aplicadas: Agents.md.

Se inspeccionaron backend y frontend, modelos, rutas, servicios, tareas, bitácoras, ventas, reportes, permisos, suscripciones y almacenamiento offline. La entrega es una auditoría de software: no modifica datos ni prescribe tratamientos o tiempos de retiro.

1. Resumen Ejecutivo

Sanidad tiene un registro operativo desarrollado, pero no garantiza todavía integridad clínica histórica, ejecución atómica ni control de animales medicados al vender. Distingue planes recurrentes, tratamientos con aplicaciones sucesivas y aplicaciones únicas. Registra responsables, animales, fechas y eventos; admite operaciones colectivas y estados sanitarios separados del estado de inventario.

Se encontraron **14 hallazgos: 8 P1 y 6 P2**. P1 indica prioridad alta por aislamiento, integridad del historial, continuidad sanitaria o controles comerciales; P2 indica defectos funcionales, de presentación o capacidades operativas incompletas. No son puntuaciones CVSS.

Los problemas principales son:

- Planes y tratamientos aceptan campos de organización/finca del cliente en sus altas; el plan también permite reasignarlos al editar. Las entradas de protocolos y el CRUD sanitario legado requieren el mismo endurecimiento.
- No hay campos ni lógica de retiro de carne/leche, ni advertencia o bloqueo sanitario asociado a esos períodos en Ventas.
- La compensación manual no equivale a una transacción: se reprodujo un tratamiento eliminado tras un fallo, con animal todavía Enfermo y evento que referencia el tratamiento eliminado. También un evento paralelo tardío que sobrevive al borrado de su aplicación.
- Un reintento posterior a un error puede duplicar aplicaciones de plan; el índice único de tratamiento/número no resuelve la idempotencia de la operación completa.
- El borrado de un plan no verifica aplicaciones históricas y deja referencias sin documento origen.
- Historial y continuidad de tratamiento se restringen a la finca actual; las operaciones retroactivas de lote usan integrantes actuales.
- Completar una tarea sanitaria no crea una aplicación real; un tratamiento nuevo sin primera aplicación no programa su primera tarea.
- Las rutas sanitarias ordinarias no comprueban vigencia de suscripción.

Controles que sí existen

- Las consultas de modelos sanitarios se filtran por organización/finca mediante el plugin. Las rutas principales exigen autenticación y permisos de rol.
- Los flujos modernos validan animales de la finca activa y especie homogénea, eliminan IDs duplicados y rechazan animales vendidos/muertos en nuevas aplicaciones. Lote completo exige lote activo y con integrantes.
- Responsable/asignado del flujo ordinario se valida contra membresía activa, rol sanitario y acceso a la finca. `registradoPor` se obtiene del actor autenticado.
- Una aplicación colectiva guarda un documento con la lista de animales y crea eventos individuales; no se limita a una nota general sin animales identificables.
- Un índice único parcial declara unicidad de organización + tratamiento + número de aplicación. La API mapea el error de duplicado de esa operación a 409. No se verificó la instalación del índice en producción.
- No hay DELETE público de TratamientoSanitario ni AplicacionSanitaria. Completar/cancelar conserva aplicaciones, y tras registrar alguna se bloquea el cambio de animales/lote del tratamiento.
- Los cambios ordinarios de estado sanitario exigen un motivo y generan evento. Terminar un tratamiento no impone automáticamente Sano: el usuario elige y el valor predeterminado es «No modificar».

Verificación y límites

`npm.cmd test` en `backend`: **242 pruebas aprobadas, 0 fallidas, 0 omitidas**, salida 0. Se ejecutaron además **14 comprobaciones aisladas**, detalladas en la sección 8, usando funciones/controladores reales y dobles de almacenamiento; las comprobaciones de altas y filtros incluyeron validación/hooks reales de Mongoose.

No se accedió a MongoDB ni a datos productivos, no se ejecutaron migraciones y no se realizaron peticiones HTTP autenticadas ni pruebas en navegador. Las carreras se reprodujeron intercalando promesas en memoria. La lectura de formularios identifica defectos del código, sin certificar su renderizado. La suite existente no cubre la totalidad de los hallazgos.

2. Catálogo Exhaustivo de Funcionalidades Activas

«Básico» significa funcionalidad ejecutable con alcance limitado; «Incompleto» señala defectos o integración faltante. No se certifica «100% Funcional» sin validación integral contra base de datos e interfaz.

Submódulo / Función	Descripción en Código	Estado Backend	Estado Frontend	Madurez (100% Funcional / Básico / Incompleto)
Plan preventivo recurrente	Actividad, producto, fecha base y frecuencia en días/semanas/meses/años	Calcula siguiente fecha y estado	Formulario y tabla de planes	Básico
Alcance del plan	Todo el ganado por especie, lote dinámico, animales específicos o identificador	Resuelve animales al aplicar	Selector de lote/animales/DIIO	Básico
Plan por edad/categoría	No hay motor por edad; etiquetas como Terneros no resuelven categoría por sí solas	Requiere IDs o identificador si no es Todo el ganado/lote	Ofrece etiquetas y selección manual	Incompleto
Alertas preventivas	Próximo dentro de siete días, Vencido antes de hoy	Recalcula en ciertas lecturas; reportes usan estado persistido	Estados, Dashboard y reportes	Básico
Tarea preventiva	Próxima aplicación genera/reprograma tarea asignada	Guarda tarea después del plan	Agenda operativa	Básico
Confirmación de plan	Registra aplicación real, actualiza base y siguiente fecha	Dos rutas equivalentes; sin idempotencia integral	Modal Registrar aplicación	Incompleto
Tratamiento clínico	Motivo, diagnóstico, producto, pauta, responsable y veterinario de referencia	Alta/edición y estados Activo/Completado/Cancelado	Formulario y detalle	Básico
Primera aplicación	Opcional al crear tratamiento	Si se registra, incrementa contador y calcula siguiente	Checkbox, desmarcado inicialmente	Básico
Inicio sin primera aplicación	Tratamiento activo con contador cero y próxima fecha null	No crea tarea inicial	Se puede registrar manualmente después	Incompleto
Aplicaciones sucesivas	Número, total, fecha real, dosis y vía	Avanza desde fecha real; cierra al completar cantidad	Modal con número y estado final opcional	Básico
Completar/cancelar	Conserva aplicaciones; finaliza/cancela tareas pendientes	Cierre manual puede tener aplicaciones faltantes	Acciones y modal de estado final	Básico
Aplicación única	Registro puntual individual o colectivo	Valida ámbito y responsable, crea eventos	Formulario dedicado	Básico
Sanidad por lote	Plan dinámico; tratamiento fija animales; aplicación guarda lista	Documento colectivo y eventos individuales/lote	Acceso desde Lotes y Sanidad	Básico
Estado sanitario	Sano/En observación/Enfermo/Recuperación, independiente de Activo/Vendido/Muerto	Cambio individual/colectivo con motivo	Inventario, ficha y Sanidad	Básico
Historia de aplicaciones	Filtros de animal, lote, especie, producto, responsable, naturaleza y fechas	Lectura de AplicacionSanitaria	Historial paginado localmente y detalle	Básico
Bitácora	Evento por aplicación/animal y cambio de estado; evento de lote	Compensación manual, sin transacción común	Ficha de animal y cronología de lote	Incompleto
Registro sanitario legado	Colección separada RegistroSanitario con CRUD	API activa, sin integración con aplicaciones modernas	Sin consumidor encontrado en la pantalla moderna	Incompleto

Submódulo / Función	Descripción en Código	Estado Backend	Estado Frontend	Madurez (100% Funcional / Básico / Incompleto)
Reporte sanitario	Cuenta aplicaciones, productos, tratamientos por estado, animales y próximas actividades	Implementado; cuenta documentos y usa estados actuales	Paneles de Reportes	Básico
Recordatorios/notificaciones	Acciones informativas y recordatorios basados en tareas	Dedupe de avisos; correo según plan	Centro de notificaciones/agenda	Básico
Protocolos	Engorde SANIDAD y banda porcina TRATAMIENTO_REPRODUCTIVO generan aplicaciones únicas	Integración PRO por servicios comunes	Formularios de protocolos	Básico
Retiro carne/leche	No hay período ni fecha de liberación sanitaria	Sin protección correspondiente en Ventas	Sin campos/advertencias de retiro	Incompleto
Dosis por peso, stock y costos	Dosis/vía son texto; no hay catálogo farmacológico vinculado, cálculo por kg ni salida de medicamento	No implementado en este flujo	Captura libre	Incompleto
Diagnóstico/alta como historia versionada	Tratamiento conserva el documento actual, no revisiones clínicas	Notificaciones de cambios; no evento clínico completo para toda transición	Sin visor de revisiones de prescripción	Incompleto
Offline de vacunación	No existe store/cola de aplicaciones	No hay contrato idempotente para captura sanitaria offline	Gestión deshabilitada sin conexión	Incompleto

Modelos y campos efectivos

Modelo	Campos relevantes y límites
PlanSanitario	Grupo textual, especie, lote, animales, animalDiio, actividad, producto, marca, dosis/vía/criterioPeso textuales, fechaAplicacion, frecuenciaCantidad/Unidad, proximaAplicacion, responsable textual, asignadoA, creadoPor, estado y observaciones. Sin edad objetivo, fecha anual independiente, retiro ni costo.
TratamientoSanitario	Animales, especie, lote, motivo obligatorio, diagnóstico opcional, producto obligatorio, dosis/vía textuales, fechaInicio, cantidadAplicaciones, intervaloDias, aplicacionesRealizadas, proximaAplicacion, fechaFin, responsable/veterinario textuales, asignadoA, estado y registradoPor. Sin retiro ni dosis estructurada.
AplicacionSanitaria	Lista de animales, especie, lote/código, fecha, producto, tipo, dosis, vía, responsable textual y usuario, motivo, observaciones, naturaleza, referencia a plan/tratamiento, número/total y registrador. Una misma dosis textual para toda la operación colectiva.
RegistroSanitario	Animal, fecha, tipo, producto, dosis, vía, responsable, próxima fecha y observaciones. Modelo legado separado, sin registro de autor propio ni puente al historial moderno.
Animal	Estado de inventario y estadoSanitario separados. No hay fecha persistida de último tratamiento ni fin de retiro. La ficha consulta tratamientos activos y último evento de cambio sanitario: animal-controller.

Todos los modelos sanitarios anteriores incorporan organización/finca y timestamps. El único historial de mediciones/aplicaciones es el conjunto de documentos; tratamiento y plan no tienen `optimisticConcurrency` activado ni registro completo de versiones de prescripción.

Mapa de APIs y artefactos

Montaje con `auth` en app.js. Frontend principal: PlanSanitario.js, FormularioPlanSanitario, FormularioTratamientoSanitario, FormularioAplicacionUnica, RegistrarAplicacionSanitaria, RegistrarAplicacionTratamiento, CompletarTratamientoSanitario, FormularioEstadoSanitario.

Ruta	Métodos y comportamiento	Fuente
`/api/sanidad`, `/api/sanidad/:id`	GET/POST y GET/PUT/DELETE del registro legado	sanidad.js, controlador

Ruta	Métodos y comportamiento	Fuente
`/api/plan-sanitario`	GET/POST	planSanitario.js
`/api/plan-sanitario/alertas`	GET de próximos/vencidos	controlador de planes
`/api/plan-sanitario/:id`	PUT/DELETE; no GET individual declarado	controlador
`/api/plan-sanitario/:id/registrar-aplicacion` y `/:id/marcar-aplicado`	PATCH; ambos ejecutan la misma función de registro real	registrarAplicacionPlan
`/api/tratamientos-sanitarios` y `/:id`	GET/POST; GET/PUT individual	rutas, controlador
`/api/tratamientos-sanitarios/:id/aplicaciones`	POST de siguiente aplicación	registrarAplicacionTratamiento
`/api/tratamientos-sanitarios/:id/completar`, `/:id/cancelar`	PATCH de cierre	controlador
`/api/aplicaciones-sanitarias`, `/:id`	GET de lista/detalle	rutas
`/api/aplicaciones-sanitarias/unica`	POST; no POST genérico en la raíz	createAplicacionUnica
`/api/animales/estado-sanitario`, `/:id/estado-sanitario`	PATCH colectivo/individual	animal.js, estadoSanitario-service
`/api/reportes/sanidad`	GET; permiso reportes.ver, sin barrera de analítica específica	reporte.js, getReporteSanidad

3. Flujo Clínico Real (Plan Sanitario vs Tratamiento Curativo vs Aplicación)

Plan preventivo

1. Se captura actividad/producto, una fecha base denominada `fechaAplicacion`, frecuencia y responsable asignable. Crear el plan **no crea evidencia de una aplicación real**. La tabla distingue `ultimaAplicacionReal`, obtenida de AplicacionSanitaria, de la fecha base del plan.
2. El hook de guardado calcula la siguiente fecha sumando días/semanas/meses/años; clasifica Próximo si está entre hoy y siete días, Vencido si es anterior a hoy, y Vigente en otro caso. No existe motor de vacunación según edad/peso ni calendario epidemiológico.
3. Se sincroniza una tarea para la **próxima** fecha calculada, no para la primera fecha base. Las notificaciones operativas dependen de tareas pendientes con fecha: sanidad-tarea-service, alertasCorreo-service.
4. Registrar aplicación resuelve los animales en ese momento: integrantes activos del lote, lista explícita de animales aún elegibles, identificador individual o Todo el ganado de la especie. Etiquetas como Terneros/Novillos/Vacas preñadas no tienen resolución automática de categoría; sin selección/identificador no encuentran animales.
5. Se cambia la fecha base del plan por la fecha de aplicación recibida, se recalcula el siguiente vencimiento y se crea AplicacionSanitaria con número `conteo + 1`. Después se completan tareas pendientes, se sincroniza la próxima y se notifica. No hay transacción que abarque el conjunto.
6. `marcar-aplicado` es alias de registrar aplicación. Aunque el enum contiene Aplicado, el hook de save recalcula el estado por fecha; el flujo normal no usa Aplicado como pausa permanente ni dispone de un campo activo para archivar el plan.

Tratamiento curativo

1. Se seleccionan animales o lote, motivo/producto, diagnóstico opcional, pauta y responsable. Las vías son texto libre, no un enum clínico ni reglas de compatibilidad.
2. Si se usa lote, se resuelve su población actual y se guarda esa lista en el tratamiento. Los animales incorporados al lote después no se agregan automáticamente. Es distinto del alcance dinámico del plan preventivo.
3. Se guarda tratamiento Activo, contador 0 y próxima fecha null. La primera aplicación es opcional; si no se registra, la siguiente fecha sigue null y no se agenda la primera tarea.
4. En cada aplicación se verifica Activo y contador menor al total. Se registra un documento con los animales originales, número siguiente y datos reales; después se actualiza el tratamiento. La próxima fecha se calcula desde la fecha real de esa aplicación más intervalo, no desde un cronograma fijo del inicio.
5. La última aplicación marca Completado, fechaFin y próxima null. Completar manualmente también está permitido antes de completar la cantidad; no fabrica aplicaciones faltantes, pero no exige un motivo específico de cierre anticipado.
6. Cancelar marca Cancelado y cierra tareas, sin cambiar el estado sanitario. No hay DELETE público del tratamiento. Editar un completado puede reabrirlo si se aumenta el total, sin flujo independiente de reapertura ni historial de prescripción anterior.

Estado sanitario: comportamiento exacto

Acción	Efecto sobre Animal.estadoSanitario
Crear plan o registrar aplicación única/preventiva	No cambia estado automáticamente
Crear tratamiento por API	Solo cambia si `actualizarEstadoSanitario === true`
Crear tratamiento desde formulario	Checkbox de cambio viene activado y propone Enfermo; puede desmarcarse o elegirse otro estado
Registrar aplicaciones intermedias	No cambia estado
Última aplicación / completar manualmente	Cambia si se envía `estadoSanitarioFinal` distinto de «No modificar»; la UI propone «No modificar»
Cancelar tratamiento	No cambia estado sanitario
Completar uno de dos tratamientos activos	No examina el otro; si se elige Sano, lo aplica a todos los animales del tratamiento cerrado
PATCH de estado	Cambio explícito con motivo y evento; no consulta otros tratamientos
Excel de Inventario	Puede escribir estadoSanitario directamente sin pasar por el servicio de cambio y su evento

Tener un tratamiento activo y estar Sano no es necesariamente una contradicción: el diseño mantiene ambas dimensiones independientes. El problema es que falta contexto/advertencia al cambiar el estado y existen vías que omiten su trazabilidad. No se propone imponer recuperación automática como regla clínica.

Aplicación colectiva, eventos y productos

Una operación para 30 animales crea **una AplicacionSanitaria con 30 IDs**, 30 eventos individuales y eventos de lote cuando corresponde. No crea 30 documentos de aplicación. Producto, dosis y vía se comparten en ese documento; no hay cantidad aplicada ni resultado diferente por animal, ni un mecanismo para marcar excepciones dentro de la confirmación de un plan completo.

El servicio crearAplicacionSanitaria guarda la aplicación, crea eventos con Promise.all y, si falla, intenta borrar eventos y aplicación. Es compensación, no sesión MongoDB. Crear/modificar/completar/cancelar un tratamiento emite notificaciones, pero no siempre un evento clínico individual de esa transición: sin aplicación y sin cambio de estado puede no quedar evento de diagnóstico en la bitácora.

Engorde SANIDAD y banda porcina TRATAMIENTO_REPRODUCTIVO llaman al mismo servicio como aplicaciones únicas: protocoloEngorde-service, protocoloPorcino-service. Las tareas sanitarias de reproducción/camadas también generan notas de bitácora al completarse, pero no equivalen a AplicacionSanitaria estructurada.

No hay cálculo de dosis por peso: `criterioPeso`, `dosis` y `viaAplicacion` son textos. No se encontró descuento de medicamento, lote farmacéutico/vencimiento, costo por aplicación ni creación automática de MovimientoFinanciero en este flujo. Los gastos generales de Sanidad en Finanzas no constituyen trazabilidad de consumo farmacológico.

4. Análisis de Periodos de Retiro e Inocuidad

No existe protección implementada de retiro de leche/carne. Los modelos inspeccionados no contienen período de retiro, fecha de liberación, especie/producto con regla de espera ni diferencia carne/leche. Tampoco hay interfaz para capturarlos.

El validador de ventas comprueba animales existentes, especie, duplicados, estado de inventario, venta previa y peso/precio. No consulta tratamientos, aplicaciones ni retiro. R10 ejecutó ese validador con animal Activo/Enfermo y recibió `valido:true`; no se ejecutó una venta real. Esto demuestra ausencia de control sanitario en ese paso, no que toda venta de animal enfermo sea clínicamente o legalmente inválida.

No se encontró un módulo de despacho de leche enlazado a aplicaciones sanitarias. Los usos de «retiro» en protocolos reproductivos se refieren a retirar dispositivos o participantes; no son tiempos de espera de alimentos.

SAN-02 (P1) registra esta brecha porque el alcance solicitado requiere evaluar si el software protege la comercialización de animales medicados. El sistema hoy no puede determinar si existe retiro activo ni garantizar liberación por cambiar el animal a Sano o completar un tratamiento.

La remediación debe modelar datos de retiro definidos por el responsable veterinario y su fuente, guardar reglas aplicadas por producto/especie/vía cuando corresponda, calcular restricciones a partir de aplicaciones reales y conciliarlas ante correcciones. Ventas debe consultar una restricción vigente y devolver la advertencia/bloqueo acordado con trazabilidad de cualquier excepción. No se proponen aquí días genéricos ni reglas farmacológicas deducidas del nombre comercial.

5. Evaluación de Riesgos e Inconsistencias (P1 y P2)

SAN-01 — P1: payloads permiten alterar ámbito y campos internos

Evidencia: create/update de plan expande req.body y luego usa Object.assign; create de tratamiento expande req.body; CRUD legado guarda/actualiza el cuerpo sin DTO. El plugin completa tenant solo si falta.

Reproducción R1: altas reales de plan, tratamiento y registro legado llegan a insertOne simulado con organización/finca ajenas, ejecutando los hooks reales. El plan puede además cambiar creadoPor y otros campos del documento al editar. El CRUD legado acepta un animal sin verificar existencia ni finca; PUT pasa directamente el cuerpo a findByIdAndUpdate, incluidos operadores de actualización si se suministran.

El tratamiento sí tiene lista de campos permitidos en edición y fija contador/estado/autor al crear. La aplicación única ordinaria construye explícitamente los campos del cliente y fija autor/naturaleza: no se atribuye a esa ruta el mismo spread inseguro. Sin embargo, los protocolos pasan `...datos.aplicacion` al servicio común, que también hace spread al persistir; deben cerrar esa entrada de campos internos y validar responsable por el contrato del protocolo.

Acción: DTO por entidad/operación, rechazo de operadores/rutas con punto en payloads, tenant y autoría del contexto, y validación de cada referencia. Las consultas filtradas no protegen un tenant aceptado al guardar.

SAN-02 — P1: sin control de retiro al comercializar

Desarrollado en la sección 4. No existe un campo informativo de retiro que esté siendo ignorado: **la capacidad no está modelada**. No presentar el módulo como protector de inocuidad ni inferir liberación por estado Sano/Completado.

Acción: definir modelo, fuentes, restricciones carne/leche y contrato con Ventas; pruebas de varias aplicaciones/productos, correcciones y última aplicación. La política clínica debe ser explícita, no deducida por el código de una dosis textual.

SAN-03 — P1: compensaciones dejan estado e historia inconsistentes

Evidencia: createTratamiento, crearAplicacionSanitaria, estadoSanitario-service.

Reproducciones: R9 fuerza fallo en sincronización de tarea después del alta y cambio sanitario: el catch elimina tratamiento/aplicaciones, pero deja Animal=Enfermo y un evento de estado referido al tratamiento borrado. R11 fuerza fallo de un evento de la operación colectiva mientras otro espera: el catch borra aplicación/eventos y luego el evento pendiente termina, quedando huérfano.

Las compensaciones pueden fallar a su vez. Restaurar snapshots con `Animal.collection.updateOne` tampoco compara versión: un rollback tardío puede sobrescribir un cambio concurrente posterior. La creación de plan y sincronización de tarea, así como cierres de tratamiento/estado/tareas, son escrituras separadas.

Acción: transacción de tratamiento/aplicación/estado/eventos/tareas según la unidad de operación; control de versión en conflictos y notificaciones posteriores a commit o mediante outbox. No usar limpieza temprana de Promise.all como garantía de que las demás escrituras terminaron.

SAN-04 — P1: reintentos y concurrencia no tienen identidad de operación

Evidencia: registrarAplicacionPlan numera con count+1 sin índice único de plan/número; las aplicaciones únicas no tienen clave idempotente. registrarAplicacion de tratamiento usa contador leído y save sin control optimista del tratamiento.

R12: error al completar tarea después de guardar aplicación de plan devuelve 400; repetir el mismo payload crea una segunda aplicación para la misma fecha. En tratamiento, si la aplicación/contador se guardan pero falla un efecto posterior, el reintento puede convertirse en la siguiente dosis; el índice único de número no lo detecta porque ya es otro número.

El índice tratamiento/número sí limita inserciones concurrentes del mismo número si está desplegado, pero no serializa editar/completar/cancelar frente a registrar, ni coordina ejecución de protocolo con aplicación. Una respuesta perdida tampoco autoriza asumir otra aplicación.

Acción: clave idempotente por acto sanitario, versión esperada de tratamiento/plan y transacción. Probar repetición tras error/red, conflicto de número y cancelación simultánea. No imponer unicidad general animal/día: puede haber actos legítimos distintos.

SAN-05 — P1: borrar plan deja aplicaciones sin origen resoluble

Evidencia: deletePlanSanitario hace findByIdAndDelete y luego cancela pendientes, sin comprobar AplicacionSanitaria. R6 verificó que no consulta dependencias antes de borrar.

Efecto: se conservan aplicaciones y sus eventos/datos principales, pero `planSanitario` queda como referencia colgante y populate devuelve null. El historial frontend puede etiquetar el origen como «Registro directo» aunque naturaleza siga siendo Plan sanitario. Las tareas históricas también pierden la referencia al plan. No hay snapshot completo de la configuración del plan para reconstruirlo.

Tratamientos y aplicaciones no tienen DELETE público. El CRUD legado sí permite borrado definitivo por Administrador. No se encontró una revisión de corrección/anulación de aplicaciones modernas; inmutabilidad sin un procedimiento de rectificación también limita la recuperación de errores.

Acción: archivar planes o responder 409 al borrado con historia; conservar origen identificable. Diseñar corrección/anulación clínica con motivo, autor y revisiones, sin borrar evidencia aplicada.

SAN-06 — P1: historial y población no siguen la fecha y finca del acto clínico

Evidencia: resolverAlcanceSanitario toma pertenencias `activo:true`; obtenerAnimalesParaPlan usa alcance actual; getAplicaciones no habilita historia entre fincas. Los tratamientos activos de la ficha animal también quedan filtrados por finca actual.

Efectos: registrar hoy una aplicación de lote de la semana pasada la atribuye a sus miembros actuales; un animal entrado después puede aparecer como tratado y uno salido puede faltar. Si un animal del tratamiento fijo fue vendido/murió, la validación de todos sus integrantes bloquea la siguiente aplicación del grupo; no existe exclusión individual con motivo. Tras traslado a otra finca, tratamientos/aplicaciones quedan en origen, la ficha destino no los reúne y la ejecución desde origen no encuentra al animal movido. R13 comprobó el filtro de finca destino en el historial por animal, usando hooks reales.

El servicio común valida lote existente y especie, pero no siempre pertenencia de los animales a ese lote: el protocolo de engorde puede enviar `datos.animales` de la finca y atribuirlos al lote del ciclo sin comprobar que sean miembros. Las aplicaciones individuales sin lote generan eventos usando loteActual, también para fechas retroactivas.

Acción: preservar cohorte aplicada y resolver pertenencia a fecha cuando se registra retroactivamente; permitir excepciones por animal con trazabilidad. Definir continuidad sanitaria entre fincas autorizadas y origen visible, sin abrir todo el historial de la organización indiscriminadamente. Comprobar alcance en todos los llamadores del servicio.

SAN-07 — P1: agenda y evidencia de aplicación pueden divergir

Evidencia: sincronizarTareaTratamiento, completarTarea, sincronizarBitacoraTarea.

- Tratamiento creado sin primera aplicación tiene proximaAplicacion=null; la sincronización cancela pendientes y no programa inicio. Es el comportamiento predeterminado del formulario. R4 lo confirmó.
- Completar una tarea Plan sanitario/Tratamiento sanitario no invoca el registro clínico ni incrementa contador ni reprograma pauta. A diferencia de IATF/engorde/bandas, esas categorías no están bloqueadas en completarTarea.
- `generaBitacora:false` no impide evento si tipo=Sanidad: la condición usa OR con el tipo de evento. R7 completó una tarea sanitaria reasignada a Trabajador: quedó Completada, produjo evento individual y **cero AplicacionSanitaria**. La agenda puede dejar de alertar aunque la aplicación no esté documentada.
- La sincronización reutiliza por clave una tarea completada/cancelada conservando su estado. Si se reprograma a esa misma clave/fecha, puede no reabrirse. No hay índice único de clave automática que garantice una sola tarea ante sincronizaciones simultáneas.

Acción: programar primera dosis a fechaInicio; separar completar logística de confirmar aplicación. Requerir el formulario clínico o una operación de ejecución asignada que registre aplicación y agenda conjuntamente. Hacer explícita la reapertura/idempotencia de tareas y respetar generaBitacora.

SAN-08 — P2: estado sanitario manual y prescripciones sin trazabilidad uniforme

Evidencia: actualizarEstadoDesdeTratamiento, estadoSanitario-service, importarAnimales.

Completar un tratamiento puede marcar Sano aunque otro siga activo: no hay consulta ni advertencia de los otros tratamientos. No se propone prohibirlo automáticamente, pero debe mostrarse el contexto y conservar una decisión explícita. Cancelar deja el estado sanitario anterior; finalizar con «No modificar» también. La UI sí permite distinguir estado sanitario y tratamiento activo en Inventario.

La importación Excel conserva `estadoSanitario` entre los datos asignados directamente a Animal y no utiliza el servicio que exige motivo/evento. El CRUD normal de animal protege la edición de ese campo, de modo que las vías no son equivalentes. Crear un animal admite estado inicial sin que esto sea una corrección clínica.

Diagnóstico/producto/pauta de tratamiento son editables con aplicaciones previas; los documentos de aplicación conservan sus propios valores, pero no existe historial de revisiones del tratamiento. Un cierre anticipado o reapertura al aumentar cantidad no requiere motivo específico. Notificaciones de acción no sustituyen una historia clínica versionada.

Acción: avisar de otros tratamientos al cambiar estado; canalizar cambios de Excel por servicio auditado; guardar motivo/revisiones de prescripción y distinguir fin completo, cierre anticipado y cancelación.

SAN-09 — P2: fechas, cantidades y alcance preventivo insuficientemente validados

Evidencia: esquemas de TratamientoSanitario, PlanSanitario, controladores y formularios.

- `cantidadAplicaciones` acepta fracciones. R3 confirmó que 1.5 aplicaciones e intervalo 1.5 pasan la validación del modelo, sin persistir en MongoDB: después de una aplicación no se completa, pero una segunda excede el total y falla validación. Frecuencias/intervalos tampoco exigen enteros ni tienen una política explícita de horas.
- No se valida coherencia de fechaInicio, fechaAplicacion y fechaFin, ni futuro, nacimiento o orden respecto de aplicaciones anteriores. Una aplicación retroactiva puede mover la próxima fecha hacia atrás; en el plan sustituye la base aun si existe una aplicación posterior.
- La suma mensual usa `Date.setMonth`: 31 de enero + un mes = 3 de marzo de 2026, no fin de febrero. Es el comportamiento ejecutado, cuya adecuación requiere una regla de calendario definida.
- Los inputs date usan ISO UTC y la visualización usa hora local; en Costa Rica una fecha de medianoche UTC puede mostrarse el día anterior. Vencimientos del modelo usan día local del servidor; otras operaciones usan UTC. No hay contrato uniforme de fecha civil/zona de organización.
- Las etiquetas Terneros/Novillos/etc. no seleccionan categoría automáticamente; un plan con solo esa etiqueta puede crearse y luego no aplicarse (R8). Cambiar especie en un PUT parcial de plan sin reenviar animales/lote no revalida todo el alcance existente.

Acción: enteros de conteo, fechas y cronología coherentes, política de calendario/zonas horarias y selección de alcance validada sobre el documento resultante. No interpretar una etiqueta o texto de dosis como regla clínica automática.

SAN-10 — P2: editor por lote y controles de interfaz desalineados

Evidencia: FormularioTratamientoSanitario envía animales=[] cuando el alcance es lote, incluso al editar; updateTratamiento compara esa lista con la guardada tras la primera aplicación.

R5: editar solo observaciones de un tratamiento por lote con aplicación previa devuelve 400 por cambio de animales. Los selectores están bloqueados en UI, pero el payload modifica la lista. Además, los catálogos cargan solo lotes/animales activos, lo que dificulta presentar referencias históricas inactivas.

Los botones Eliminar plan aparecen a todo gestor sanitario, incluyendo Veterinario/Encargado, aunque DELETE solo admite Administrador. La API sí deniega; no se observó escalada de privilegios por ese botón. Editar tratamiento Cancelado también se ofrece y el backend lo rechaza.

El detalle de aplicación usa el objeto ya listado y presenta solo parte de sus datos; no muestra un historial de correcciones ni todos los campos de autoría/observaciones. Los listados descargan todos los registros; la paginación es del cliente. Filtros producto/responsable se interpretan como expresiones regulares en backend sin escape ni límites específicos, por lo que símbolos pueden cambiar la búsqueda o generar errores.

Acción: preservar alcance al editar, normalizar referencias y mostrar históricos seleccionados; permisos específicos para borrar/editar; consulta literal y paginación de servidor; detalle de evidencia clínica suficiente para revisión.

SAN-11 — P2: ruta sanitaria legada y herramientas de migración no reconciliadas

Evidencia: sanidad-controller, migrarAplicacionesPlanesSanitarios, sincronizarTareasSanidad.

El CRUD RegistroSanitario continúa expuesto, pero no crea AplicacionSanitaria, EventoAnimal ni actualiza estados/tareas; sus registros no aparecen en el reporte de aplicaciones moderno. Aun así, se cuentan como dependencia para impedir borrar animales/compras. La historia queda fragmentada según la vía usada.

Los scripts sanitarios inspeccionados consultan modelos aislados sin establecer contexto de organización/finca; con el plugin actual las consultas sin contexto se filtran a la organización centinela `000000000000000000000000`. Por inspección, no recorren normalmente los datos de los clientes. **No se ejecutaron** scripts ni se inspeccionó su configuración privada.

La migración de aplicaciones también toma la fecha base de un plan y, si no hay eventos legados, resuelve animales actuales para crear una aplicación histórica. Ese dato no demuestra por sí solo que se aplicó el producto a dichos animales; no debe usarse como fuente automática de evidencia clínica sin conciliación.

Acción: decidir retirada o adaptación del API legado, inventariar fuentes y migrar con contexto explícito por tenant. Requerir evidencia para materializar aplicaciones; ejecutar simulación y reconciliación antes de aplicar cambios históricos.

SAN-12 — P2: reportes mezclan unidades y dependen de estado persistido

Evidencia: getReporteSanidad, refrescarEstado, Reportes.

`totalAplicaciones` y productos más aplicados cuentan documentos: una vacunación de 30 animales cuenta **1**, no 30 dosis ni volumen de producto. Animales más tratados sí usa unwind de la lista, solo para naturaleza Tratamiento. La unidad no queda suficientemente diferenciada bajo «aplicaciones» y «uso real».

Tratamientos por estado filtra fecha de inicio y muestra estado actual, no el que tenían al cierre histórico del período. Próximos tratamientos/planes son una agenda actual, sin el mismo rango temporal. No se calcula incidencia epidemiológica, tasa por población

expuesta, retiro ni costo veterinario por caso.

El reporte consulta planes con estado guardado Próximo/Vencido; no recalcula por fecha. Un plan almacenado Vigente que venció puede faltar hasta que otra lectura lo refresque (R14). GET de planes/alertas sí puede guardar estados, por lo que distintos endpoints dependen del orden de consulta. Los recordatorios por tareas son otro flujo y no corrigen necesariamente ese estado del plan.

Acción: separar número de actos, animales intervenidos y cantidad de producto cuando exista; etiquetar agenda actual frente a período histórico; calcular alertas por fechas y evitar que un GET previo sea requisito de exactitud.

SAN-13 — P1: no se valida suscripción vigente en las operaciones sanitarias

Evidencia: auth exige organización activa y calcula planVigente, pero no bloquea por sí mismo la suscripción suspendida/cancelada. Rutas de sanidad, planes, tratamientos, aplicaciones y PATCH de estado sanitario no usan `requirePlanVigente` ni `requireFeature`.

R2 recorrió permiso de Veterinario y handler real de POST legado con planVigente=false y obtuvo 201. La ausencia de comprobación se verificó estáticamente también en los otros controladores. No fue una prueba HTTP de autenticación completa. Una organización inactiva sí es rechazada por auth; no debe confundirse con organización activa y suscripción no vigente.

Los protocolos PRO sí tienen requireFeature; esto no protege sus equivalentes ordinarios. El listado de lotes puede denegar por otro chequeo de plan y bloquear la carga del frontend, pero no reemplaza validación en las APIs sanitarias directas.

Acción: definir lectura histórica y escritura bajo suspensión, aplicar vigencia a todas las entradas, incluidas heredadas y estado sanitario. No limitar sanidad básica a un plan superior si el contrato la ofrece en Esencial.

SAN-14 — P2: sin captura estructurada de vacunación offline

Evidencia: offlineStorage no tiene aplicaciones/tratamientos; offlineSync solo completa tareas o crea notas. ListaUsuario pone Sanidad en solo lectura sin conexión.

Si se aplican 30 vacunas sin señal, no hay cola sanitaria para registrar las 30 intervenciones con fecha real, responsables y producto. Completar una tarea offline puede sincronizar una tarea, pero no una AplicacionSanitaria: ver SAN-07. El precache del sitio no equivale a disponer de datos/formularios clínicos persistentes.

Acción: definir captura offline por acto sanitario, lista de animales, responsables autorizados y snapshots, con idempotencia y conflictos al reconectar. Priorizar SAN-03/SAN-04/SAN-06 antes de permitir reintentos automáticos de aplicaciones.

6. Matriz de Permisos por Rol

Fuentes: permisosRoles, rutas sanitarias, usuarioAsignable-service y permisos frontend.

Capacidad ordinaria	Administrador	Encargado	Veterinario	Trabajador	Consulta	Contador
Ver módulo e historial (`sanidad.ver`)	Sí	Sí	Sí	No	Sí	No
Crear/editar planes y tratamientos (`sanidad.gestionar`)	Sí	Sí	Sí	No	No	No
Registrar aplicación única/de plan/de tratamiento	Sí	Sí	Sí	No	No	No
Completar/cancelar tratamiento	Sí	Sí	Sí	No	No	No
Cambiar estado sanitario individual/colectivo	Sí	Sí	Sí	No	No	No
Eliminar plan/registro legado (`sanidad.eliminar`)	Sí	No	No	No	No	No
Elegible como responsable/asignado de Sanidad	Sí	Sí	Sí	No	No	No
Ver reporte `/reportes/sanidad` (`reportes.ver`)	Sí	No	No	No	Sí	Sí

Veterinario: puede introducir diagnóstico y tratamiento, registrar aplicaciones y cambiar estado. No hay firma de prescripción exclusiva de Veterinario: Administrador y Encargado pueden hacer lo mismo. El campo `veterinario` es texto de referencia, no identidad profesional validada. Que Veterinario no acceda al reporte sanitario dedicado es una decisión/incongruencia de roles que debe revisarse según el producto.

Trabajador: el módulo ordinario no le permite confirmar vacunas ni modificar/eliminar diagnóstico. Tampoco se lo puede seleccionar como responsable Sanidad. Sin embargo, Tareas permite al gestor reasignarle una tarea general y él puede completarla; eso no registra la aplicación. En PRO/Premium puede ejecutar ciertos pasos de protocolo asignados, incluidos pasos sanitarios según las reglas del protocolo. Es una autorización puntual: engorde, banda porcina.

Control real de finca: el responsable ordinario debe tener membresía/usuario activos, rol admitido y acceso a la finca. La autenticación también verifica acceso a la finca seleccionada: accesoFinca-service. Esto funciona en lectura/selección, pero no neutraliza el payload de tenant al persistir descrito en SAN-01.

7. Alineación con el Modelo SaaS y los 4 Planes

Los precios mencionados en las instrucciones no coinciden con los defaults actuales de planes.js. El repositorio declara **6/12/20/35 USD mensuales**, configurables mediante variables de entorno. No se verificaron precios efectivos de producción ni configuración privada.

Capacidad / default mensual	Esencial — 6 USD	Gestión — 12 USD	Pro — 20 USD	Premium — 35 USD
Planes, tratamientos, aplicaciones y estado sanitario ordinarios	Disponibles por rol	Disponibles por rol	Disponibles por rol	Disponibles por rol
Reporte sanitario actual	Disponible con reportes.ver	Igual	Igual	Igual
Centro de alertas	Sí	Sí	Sí	Sí
Correos operativos	No	No	Sí	Sí
Analítica productiva general	No	Sí	Sí	Sí
Analítica económica general	No	No	Sí	Sí
Protocolos engorde/porcinos con acciones sanitarias	No	No	Sí	Sí
Consolidación sanitaria entre fincas en reportes	No	No	No	Sí
Retiro, dosis automática, costo clínico estructurado y captura sanitaria offline	No implementados	No implementados	No implementados	No implementados

No hay feature sanitaria específica que segmente el CRUD ordinario; se controla por roles. El reporte actual es básico y no usa guard de analítica productiva: no se encontró un cálculo avanzado de incidencia o costo clínico que esté filtrándose a Esencial. La consolidación Premium cuenta aplicaciones por finca y tratamientos activos: reporteMultiFinca-service; tampoco calcula retiro o farmacoeconomía.

El valor básico es documentar lo prescrito y aplicado y recordar lo pendiente. Integridad del historial, aislamiento y una eventual restricción sanitaria acordada no deben depender de que el usuario pague por estadísticas. Como evolución separada del alcance actual, Gestión podría incorporar cobertura/incidencia con denominadores adecuados y Pro costos por tratamiento conciliados con consumo; no debe presentarse como existente ni inferirse de los campos textuales.

La falta de vigencia en las rutas ordinarias es SAN-13. Los campos de especie también deben validarse conforme al plan contratado en todas las altas; hoy el alcance de animales limita indirectamente las operaciones modernas, pero un plan Todo el ganado puede catalogarse con otra especie sin un guard explícito de especie.

8. Plan de Acción Priorizado

Orden de corrección y aceptación

ID	Prioridad	Trabajo concreto	Criterio de cierre
SAN-01	P1	DTO/tenant/autoría/referencias en CRUD y protocolos	Alta/edición con tenant ajeno no mueve ni inserta documentos fuera de contexto; la aplicación única ordinaria sigue funcionando
SAN-02	P1	Modelo de retiro y contrato con Ventas	Aplicaciones con restricción definida impiden/avisan comercialización según política, con fechas y fuente trazables; no se libera por marcar Sano
SAN-03	P1	Atomicidad clínica con tareas/eventos y control de versión	Fallo después de cambiar estado no deja tratamiento eliminado con animal modificado; eventos paralelos no quedan huérfanos
SAN-04	P1	Idempotencia y transición concurrente	Misma operación reintentada retorna la misma aplicación; registro/cancelación simultáneos resuelven conflicto sin perder evidencia

ID	Prioridad	Trabajo concreto	Criterio de cierre
SAN-05	P1	Archivar plan o bloquear DELETE histórico; rectificaciones auditadas	Plan con aplicaciones devuelve 409 o queda archivado con referencia resoluble; no desaparece evidencia de aplicación
SAN-06	P1	Historia autorizada entre fincas y cohorte a fecha	Animal trasladado mantiene acceso permitido a su historia; lote retroactivo usa integrantes reales; protocolo no trata animales ajenos al lote sin autorización explícita
SAN-07	P1	Primera tarea y confirmación sanitaria ligada a aplicación	Tratamiento sin primera aplicación crea tarea de inicio; completar logística no se presenta como evidencia clínica; ejecución asignada registra aplicación y pauta
SAN-08	P2	Estado sanitario y revisión de prescripción auditados	Excel genera motivo/evento; cierre con otros tratamientos muestra contexto; cambios de pauta conservan versión y motivo
SAN-09	P2	Fechas civiles, enteros y alcance validado	Rechaza 1.5 aplicaciones; calendario fin de mes tiene regla probada; fecha futura/invertida y alcance vacío reciben tratamiento explícito
SAN-10	P2	Editor por lote, permisos y lectura de historia	Editar observación tras primera aplicación no cambia integrantes; solo Administrador ve borrado; referencias históricas y autoría se presentan correctamente
SAN-11	P2	Reconciliar legado y herramientas de migración	Toda vía soportada produce evidencia consultable; simulación por tenant encuentra registros reales y no inventa aplicaciones a partir de una fecha de plan
SAN-12	P2	Semántica de reporte y vencimientos por fecha	1 acto/30 animales se distingue claramente; un plan vencido aparece sin abrir antes el listado de planes; período histórico no se confunde con agenda actual
SAN-13	P1	Guard de vigencia en entradas ordinarias	Organización activa con suscripción suspendida no puede mutar por rutas directas; política de lectura histórica queda definida
SAN-14	P2	Captura offline idempotente y conflictos	30 vacunaciones capturadas sin señal conservan fecha/animales y se sincronizan una sola vez, o muestran conflictos recuperables

Antes de cualquier migración, diagnosticar aplicaciones duplicadas, referencias de planes/tratamientos eliminados, estados sin evento, eventos sin aplicación, contadores discordantes, tratamientos activos sin tarea, registros legados y historia distribuida entre fincas. Conservar valores originales y motivos de reconciliación; no recalcular silenciosamente historia clínica.

Comprobaciones aisladas ejecutadas

Se ejecutaron scripts efímeros con `node -`. Los modelos/controladores y funciones probadas son reales; consultas, escrituras, notificaciones o sincronizadores externos se sustituyeron por dobles cuando correspondía. No se afirmaron escrituras o conflictos en MongoDB real.

Prueba	Resultado observado
R1 — Altas/tenant	Plan, tratamiento y registro legado conservan tenant ajeno hasta insertOne simulado con hooks/validación reales
R2 — Suscripción y ruta legada	Permiso Veterinario y POST legado responden 201 con planVigente=false; no validan animal referenciado
R3 — Conteo/calendario	1.5 aplicaciones e intervalo 1.5 pasan validación; contador 2 falla; fecha año 2999 es admitida; 31 enero + mes da 3 marzo
R4 — Primera tarea	Tratamiento Activo con próxima null llama cancelación de pendientes, sin programar inicio
R5 — Editor por lote	Payload de edición con mismo lote y animales=[] es rechazado tras primera aplicación aunque solo se busque corregir observación
R6 — Borrado plan	Controlador elimina sin consultar aplicaciones históricas y luego cancela tareas
R7 — Tarea sanitaria	Trabajador completa tarea asignada; se crea evento aun con generaBitacora=false, sin registrar AplicacionSanitaria
R8 — Categoría del plan	Plan solo con grupo Terneros devuelve lista vacía de animales

Prueba	Resultado observado
R9 — Fallo posterior al cambio de estado	Error de tarea elimina tratamiento pero conserva Enfermo y evento con referencia al tratamiento eliminado
R10 — Venta	Validador real acepta animal Activo/Enfermo; no ejecuta consulta de retiro/tratamiento; no se realizó venta
R11 — Compensación paralela	Tras fallar un evento y borrar aplicación, otro evento pendiente termina y queda sin aplicación origen
R12 — Reintento de plan	Error de tarea posterior a aplicación devuelve 400; repetir payload genera aplicación número 2 para misma fecha
R13 — Traslado e historial	Hook real añade finca destino al historial por animal y excluye aplicación sintética en finca origen
R14 — Reporte	Consulta filtra estado persistido Próximo/Vencido; no recalcula para un plan guardado Vigente aunque su fecha ya venció

La suite existente contiene sanidad-lotes.test.js, con pruebas de campos de lote, resolución de integrantes, rechazo de lote vacío y metadatos de evento. Otras pruebas cubren dependencias de animales y permisos transversales. Deben añadirse pruebas de integración clínica, fallos, concurrencia, tenant, ventas y navegador para los criterios anteriores, y volver a ejecutar toda la suite al remediar.

Estado de entrega: auditoría completada; remediaciones pendientes. Solo se añadió este informe en esta fase.